



武汉理工大学

《企业级应用软件设计与开发》期末大作业报告

Agentic RAG -- 智能知识库助手

课程名称: 企业级应用软件设计与开发

项目名称: Agentic RAG -- 智能知识库助手

方向: 方向一: Agentic AI 原生开发

学号: 2025302966

姓名: 王继烽

专业: 计算机技术

指导教师: 戚欣

提交日期: 2026 年 6 月 22 日

目 录

一、选题背景与设计思想 - 1 -

1.1 问题定义..... - 1 -

1.2 项目价值与创新点..... - 1 -

1.3 技术路线选择..... - 2 -

二、Specs 规格文档..... - 3 -

2.1 Product Spec（产品规格书） - 3 -

2.2 Architecture Spec（架构规格书）..... - 3 -

2.3 API Spec（接口规格书） - 4 -

三、系统架构与设计 - 5 -

3.1 系统总体架构..... - 5 -

3.2 Agent 交互流程..... - 5 -

3.3 数据流设计..... - 6 -

四、关键实现与代码展示 - 8 -

4.1 Agent 核心循环..... - 8 -

4.2 工具定义与 Function Calling..... - 8 -

4.3 配置管理与安全设计..... - 9 -

五、测试与评估 - 10 -

5.1 功能测试..... - 10 -

5.2 Agent 行为评估.....	- 10 -
5.3 测试结果.....	- 10 -
六、系统升级与扩展	- 12 -
6.1 可扩展架构设计.....	- 12 -
6.2 下一阶段计划.....	- 12 -
6.3 AI 能力演进路径	- 12 -
七、课程总结	- 14 -
7.1 个人收获.....	- 14 -
7.2 工程思维转变.....	- 14 -
7.3 对课程的建议.....	- 15 -

一、选题背景与设计思想

1.1 问题定义

在信息爆炸的时代，个人和组织面临严重的知识管理困境：大量文档（报告、论文、手册、会议记录等）以非结构化形式存储，传统的关键词搜索无法理解语义，用户需要花费大量时间手动翻阅、筛选和归纳信息。

传统企业级知识管理系统的痛点包括：（1）搜索引擎基于关键词匹配，缺乏语义理解；（2）搜索结果需要人工二次加工才能得到答案；（3）不支持多轮追问和上下文理解；（4）答案来源不可追溯，无法验证准确性。

本项目的核心问题是：如何构建一个基于 **AI Agent** 的智能知识库助手，让用户可以通过自然语言对话的方式，快速获取文档中的精准信息，同时确保答案可追溯、可验证。

1.2 项目价值与创新点

本项目的核心价值主张是[从搜索到对话]：将用户与知识库的交互方式从传统的检索-浏览模式，转变为自然语言对话模式。**Agent** 自动完成检索、分析、综合推理的全过程，用户只需提问即可获得精准答案。

创新点包括：

- **Agentic RAG 架构**：将检索增强生成(RAG)与 **AI Agent** 的多步推理能力结合，**Agent** 自主决定何时检索、检索什么、如何综合信息。
- **工具化设计**：采用 **Function Calling** 机制，**Agent** 可调用多种工具(语义搜索、文档清单、上下文获取)，形成灵活的推理链。
- **多轮记忆**：基于滑动窗口的对话记忆管理，支持上下文理解和自然追问，实现类人对话体验。
- **可观测性**：**Agent** 思考过程完全透明，用户可查看每一步的工具调用和推理结果。

- 零运维向量库：采用 ChromaDB 嵌入式数据库，无需部署外部服务，一键启动即可使用。

1.3 技术路线选择

在技术选型上，本项目遵循以下原则：优先成熟稳定的开源方案；优先 LLM 生态最完善的 Python 语言；优先本地化方案以降低外部依赖。

层级	技术选型	选型理由
LLM	DeepSeek Chat API	性价比高的国产大模型，OpenAI 兼容接口
Embedding	BAAI/bge-small-zh-v1.5	本地运行，中英双语，512 维，无需 API 费用
Agent 框架	LangGraph	显式状态图，ReAct 模式，可观测性优秀
向量数据库	ChromaDB	嵌入式设计，零运维，持久化存储
UI	Streamlit	最快构建聊天界面的 Python 框架
文档解析	PyPDF + python-docx	支持 PDF/TXT/MD/DOCX 多格式
容器化	Docker	一键部署，环境隔离

表 1-1：技术栈选型总览

二、Specs 规格文档

SDD (Specification-Driven Development, 规格驱动开发) 是本课程的核心方法论。在编写任何代码之前, 先完成三份规格文档: **Product Spec** (产品规格书)、**Architecture Spec** (架构规格书)、**API Spec** (接口规格书), 确保设计先行、文档驱动。

2.1 Product Spec (产品规格书)

产品规格书定义了系统的目标用户、功能需求和非功能需求。核心用户故事包括: **US1** 上传文档并提问、**US2** 多轮追问、**US3** 知识库管理。MVP 阶段覆盖了 7 项核心功能: 文档上传与索引、语义检索、**Agent** 问答、多轮对话、来源引用、知识库管理、**Agent** 思考过程展示。

非功能性需求强调安全性 (**API Key** 环境变量管理)、可观测性 (思考链可视化)、可扩展性 (工具接口标准化) 和性能 (单次问答 < 30s)。

2.2 Architecture Spec (架构规格书)

架构规格书描述了系统的四层架构: **UI Layer** (Streamlit 聊天界面)、**Agent Layer** (LangGraph ReAct Agent)、**RAG Layer** (文档加载/嵌入/向量存储)、**Infrastructure Layer** (配置/API/Docker)。

核心设计决策包括: 选择 **LangGraph** 而非 **LangChain AgentExecutor**, 因为 **LangGraph** 提供显式状态图, 更可控、更可观测; 选择 **ChromaDB** 而非 **Pinecone/Weaviate**, 因为嵌入式设计降低运维成本; 选择本地 **BGE** 嵌入模型而非调用 **API**, 因为 **DeepSeek** 不提供 **Embedding API**, 且本地方案零费用。

2.3 API Spec (接口规格书)

API 规格书定义了 Agent 的 3 个核心 Tool 的接口规范，包括参数 Schema、返回值格式和示例。三个 Tool 分别为：search_knowledge_base (语义检索)、list_documents (列出已索引文档)、get_context (获取完整上下文)。每个 Tool 都有完整的 JSON Schema 定义，遵循 OpenAI Function Calling 标准。

此外，API Spec 还定义了内部 Python API (Loader/Embedder/VectorStore/Agent 模块的接口) 以及环境变量规范 (9 个配置项，含必需和可选)。

三、系统架构与设计

3.1 系统总体架构

系统采用分层架构设计，各层职责清晰、接口明确。整体架构如下：



图 3-1：系统四层架构总览

3.2 Agent 交互流程

Agent 采用 ReAct (Reasoning + Acting) 模式，通过 LangGraph 状态图编排推理与行动的交替循环。一次典型的问答流程为：用户提问 -> Agent 决策 (需要检索) -> Tool 执行语义检索 (返回 Top-4 相关片段)

-> Agent 综合推理 -> 生成带来源引用的最终回答 -> 存入 Conversation Memory 支持后续追问.

LangGraph 编译后的状态图包含两个核心节点：agent 节点 (LLM + 绑定 Tools) 和 tools 节点 (ToolNode 执行器). 条件边 `should_continue` 根据 Agent 输出判断是否需要继续调用工具：有 `tool_calls` -> tools 节点, 无 -> END.



图 3-2: Agent ReAct 交互流程

3.3 数据流设计

系统包含两条核心数据流:

(1) 文档摄入流 (Ingestion Pipeline): 文件上传 -> 格式检测 -> 文档加载 (PyPDF/Docx2txt/TextLoader) -> 递归分块 (`chunk_size=1000`,

overlap=200) -> 向量化 (BGE-small-zh-v1.5, 512 维) -> ChromaDB 持久化.

(2) 问答流 (Query Pipeline): 用户提问 -> Agent Node (LLM 推理) -> 决策 (Tool Call) -> ChromaDB 相似度搜索 -> Top-K 相关片段返回 -> Agent Node (综合推理) -> 生成回答 + 引用来源 -> 存入 Conversation Memory.

数据流阶段	输入	输出
文档加载	PDF/TXT/MD/DOCX 文件	Document 对象列表
文档分块	Document 对象	分块后 Document 对象 (~1000 字/块)
向量化	文本字符串	512 维浮点向量
语义检索	查询文本	Top-K 相关 Document (含相似度分数)

表 3-1: 各数据流阶段的输入输出

四、关键实现与代码展示

4.1 Agent 核心循环

Agent 核心基于 LangGraph 的 StateGraph 构建。状态定义为 messages 列表，使用 add_messages 归并器自动累积对话。LLM 实例通过 ChatOpenAI（指向 DeepSeek 端点）创建，绑定 3 个自定义 Tool，temperature 设为 0.3 以保证事实准确性。

```
def create_agent() -> StateGraph:
    llm = create_llm()
    llm_with_tools = llm.bind_tools(ALL_TOOLS)

    def agent_node(state):
        response = llm_with_tools.invoke(state['messages'])
        return {'messages': [response]}

    def should_continue(state):
        last = state['messages'][-1]
        if last.tool_calls:
            return 'tools'
        return '__end__'

    workflow = StateGraph(AgentState)
    workflow.add_node('agent', agent_node)
    workflow.add_node('tools', ToolNode(ALL_TOOLS))
    workflow.set_entry_point('agent')
    workflow.add_conditional_edges('agent', should_continue, {
        'tools': 'tools', '__end__': END
    })
    workflow.add_edge('tools', 'agent')
    return workflow.compile()
```

代码 4-1: Agent 核心图构建 (src/agent/core.py)

4.2 工具定义与 Function Calling

系统定义了 3 个 Agent Tool，每个 Tool 都使用 LangChain 的 @tool 装饰器，自动生成符合 OpenAI Function Calling 规范的 JSON Schema。Tool 是 Agent 与知识库交互的唯一桥梁。

```
@tool
def search_knowledge_base(query: str) -> str:
    """在知识库中语义搜索相关文档片段。"""
```

```

results = similarity_search(query, k=4)
for i, doc in enumerate(results, 1):
    score = doc.metadata.get('score', 'N/A')
    source = doc.metadata.get('source', 'unknown')
    parts.append(f'[{i}] source: {source} | score: {score}')
return '\n'.join(parts)

```

代码 4-2: search_knowledge_base Tool (src/agent/tools.py)

每个 **Tool** 返回结构化的文本结果，包含来源文件名和相似度分数。**Agent** 基于这些结果进行综合推理并生成最终回答。**Tool** 的设计原则是单一职责：**search** 负责检索，**list_documents** 负责列举，**get_context** 负责获取上下文。

4.3 配置管理与安全设计

遵循课程学术纪律要求，所有 **API Key** 通过环境变量管理，不硬编码在代码中。项目提供 **.env.example** 模板文件，用户复制为 **.env** 后填入自己的 **Key**。**.env** 文件已被 **.gitignore** 排除，确保密钥不会提交到 **Git** 仓库。

```

# .env 文件结构（实际值不提交到仓库）
DEEPSEEK_API_KEY=sk-xxxxxxx
LLM_MODEL=deepseek-chat
EMBEDDING_MODEL=BAAI/bge-small-zh-v1.5
CHUNK_SIZE=1000
MAX_HISTORY_TURNS=10

```

代码 4-3: 环境变量配置示例 (.env.example)

配置模块 (**src/config.py**) 负责加载 **.env** 文件并提供 **Config** 单例。启动时自动验证必需变量（如 **DEEPSEEK_API_KEY**），缺失时抛出明确错误提示。

五、测试与评估

5.1 功能测试

项目包含集成测试套件 (tests/test_basic.py)，覆盖 4 个核心模块：Config 加载、文档分块、Vector Store CRUD 操作、Agent 图编译。测试使用真实 API Key 和本地嵌入模型，验证端到端功能正确性。

测试用例	测试内容	结果
Config Loading	验证 .env 加载、路径解析、默认值	通过
Document Chunking	验证文档分块数量、元数据保留	通过
Vector Store CRUD	验证添加、检索、列表、删除、清空	通过
Agent Compilation	验证 LangGraph 状态图编译成功	通过

表 5-1: 集成测试结果 (4/4 通过)

5.2 Agent 行为评估

Agent 行为评估从三个维度进行：检索质量（返回文档的相关度和排序准确性）、推理质量（Agent 是否正确使用检索结果进行综合推理）、以及对话连贯性（多轮追问时是否正确理解上下文）。

评估方法采用人工验证 + 自动化测试结合的方式。测试用例涵盖：单文档问答、多文档交叉检索、追问上下文理解、空知识库边界处理等场景。Agent 在典型场景下能够正确执行 ReAct 循环，调用合适的 Tool，并生成有来源引用的回答。

5.3 测试结果

全部 4 项集成测试通过 (4/4)。Vector Store 操作测试验证了完整的 CRUD 生命周期：添加 3 个文档片段后检索返回正确的相关结果，相似

度分数合理，清空操作确认数据完全删除。Agent 编译测试确认 LangGraph 图结构正确，节点和边配置无误。

性能方面，本地 BGE 嵌入模型在 CPU 上完成 512 维向量化，嵌入速度约 6700 it/s，单次文档分块 + 向量化 + 入库全过程 < 5 秒。

Agent 单次问答（含检索 + LLM 推理）通常在 5-15 秒内完成，满足 < 30 秒的性能要求。

六、系统升级与扩展

6.1 可扩展架构设计

系统架构从设计之初就考虑了可扩展性。核心扩展点包括：Tool 接口标准化 -- 新增 Tool 只需用 @tool 装饰器定义函数并加入 ALL_TOOLS 列表，Agent 自动获得新能力；LLM 可替换 -- 基于 LangChain 的 ChatOpenAI 抽象，切换到 Anthropic/OpenAI/Ollama 只需修改配置；多 Collection 支持 -- ChromaDB 天生支持多 Collection，可实现按项目/用户隔离的知识库。

6.2 下一阶段计划

v0.2 版本计划增加以下功能：

优先级	功能	技术方案
P0	MCP 协议集成	通过 MCP Server 外挂工具，Agent 可访问外部 API/数据库
P0	REST API	FastAPI 提供 HTTP 接口，支持第三方系统集成
P1	用户认证	JWT + 多用户隔离的知识库 Collection
P1	云部署	Docker + 云服务器，提供公网可访问 URL
P2	评估体系	RAGAS 评估框架 + 检索质量 Benchmark

表 6-1：v0.2 版本迭代计划

6.3 AI 能力演进路径

中长期来看，系统的 AI 能力演进路径分为三个阶段。第一阶段（当前 MVP）：单 Agent + RAG，实现基础的文档问答能力。第二阶段（v0.2-v0.5）：多 Agent 协作 -- 引入 LangGraph 的多 Agent 编排，不同 Agent 负责检索、推理、摘要等不同任务，协同完成复杂查询。

第三阶段 (v1.0): 自主 Agent -- Agent 具备长期记忆 (跨会话)、主动学习 (从用户反馈中改进)、以及自主决策能力 (自动更新知识库、自动优化检索策略)。最终目标是打造一个真正智能的企业知识管理伙伴,而不仅仅是一个问答工具.

七、课程总结

7.1 个人收获

通过本次期末大作业，我从零构建了一个完整的 AI Agent 系统，经历了从需求分析、规格设计、架构选型、代码实现到测试验证的完整工程闭环。最大的收获是理解了规格驱动开发(SDD)的价值：在写第一行代码之前先完成 Product/Architecture/API 三份 Spec，让后续的编码工作有据可依、有章可循。

技术层面，我对以下领域有了深入理解：(1) RAG 系统的完整 Pipeline -- 从文档加载、分块策略、嵌入模型选择到向量检索的端到端流程；(2) LangGraph 的 Agent 编排 -- 状态图、条件路由、Tool 绑定等核心概念的实际应用；(3) Function Calling 机制 -- 如何设计 Tool 的接口让 LLM 正确调用；(4) 工程安全意识 -- API Key 管理、.gitignore 配置、环境变量注入等最佳实践。

7.2 工程思维转变

本课程最大的思维转变是[从代码编写者到智能体编排者]的角色升级。传统软件开发中，开发者编写确定的逻辑流程；而在 AI Agent 开发中，开发者的角色转变为：定义 Agent 的能力边界 (Tools)、设计推理框架 (ReAct/LangGraph)、管理 Agent 的记忆和状态，然后把具体的推理过程交给 LLM。

这种转变意味着新的工程挑战：如何设计 Prompt 引导 Agent 正确使用工具？如何在灵活性和可控性之间平衡？如何评估 Agent 的行为质量？这些问题没有标准答案，需要在实践中不断探索和迭代。

7.3 对课程的建议

课程内容充实，理论与实践结合紧密。SDD 方法论和 Milestone 分阶段交付的设计非常好，避免了一次性提交的瀑布式压力。两个方向的选题设计给了学生灵活的选择空间。

建议方面：(1) 可以增加课堂上的代码 Review 环节，让同学们互相学习优秀的设计模式；(2) 可以提供一個最小可运行的参考实现，降低起步门槛；(3) MCP 协议是今年的热点技术，建议在课程中增加相关实验内容。